

Ревью прикладного кода

Соблюден [стандарт](#)

Как подготовиться к review

Давайте представим, что вам нужно отдать свой проект во внешнюю поддержку (аутсорсинг).

В таком случае все должно быть максимально прозрачно. Держите эту идею в голове, следуя простым рекомендациям. И перед отправкой кода на review, проверьте его самостоятельно.

Модульный подход к review

Модульное review подразумевает поэтапную проверку. Отдельные части программы отправляются на review по мере их выполнения. Если на момент готовности проекта его этапы приняты, то фактически останется только показать работу программы в целом.

Основные требования к коду

Соответствует нашему стилю и [стандарту](#) (форматирование, расположение скобок, отступы, длина)

Наглядные имена переменных, функций

1. Использовать длинные наглядные имена – ЭтоНалоговыйСчет, чтобы без комментария понять смысл функции, переменной. Исключение локальные переменные, типа итераторов цикла, временных переменных или результата исполнения
2. Избегать одинаковых названий функции разного действия
3. Использовать смысловые в названии is/Это, get/Получить; существительное или глагол.
4. Использовать только русские названия функций.

Без дублирования

1. Объединение дублирующего кода позволяет улучшить структуру кода и уменьшить его объём.
2. Упрощение и удешевление поддержки кода в будущем.

Но, если объединение сделает код менее очевидным и понятным, откажитесь от объединения.

Минимум глобальных переменных

1. Глобальные переменные ухудшают читаемость кода (в каком-то конкретно взятом месте непонятно, нужна ли какая-то конкретная глобальная переменная, или нет).
2. Глобальные переменные приводят к трудноуловимым ошибкам.

Примеры: нежелательное изменение её значения в другом месте, ошибочное использование глобальной переменной для промежуточных вычислений из-за совпадения имен.

А давайте одежду дома складывать прямо на пол в любом месте квартиры: ведь 30 кв. см ящика в комод - это те же 30 кв. см поверхности, что и на полу, только в пределах ящика. И гаражи для машин не нужны, можно на улице ставить.

Без мертвого/закомментированного кода

Если чувствуете, что мертвый или закомментированный код больше не потребуется, но вы не хотите его удалять, потому что точно не знаете, понадобится он в будущем или нет — УДАЛИТЕ ЕГО ПРЯМО СЕЙЧАС! Хранить код — это работа системы контроля версий, а не комментарии!

«Совершенство достигнуто не тогда, когда нечего добавить, а тогда, когда нечего отнять» (Антуан де Сент-Экзюпери)

Без элементов логирования

Служебных функций ЗаписатьВЛог, СообщитьВКонсоль не должно быть в коде перед выкладкой в репозиторий.

Без заикливания

Устанавливайте размер цикла или указывайте условия завершения.

Количество строк в методе не превышает 25-50 строк

Проще читать и редактировать код, если он помещается на одну страницу. При этом код метода должен выполнять определенную задачу. Если в функции блоки выполняют разные задачи или хочется выделить их комментариями – лучше делить на методы.

Безопасность

Обработка неверных значений параметров функции

Без eval

Выполняет произвольное выражение, переданное в виде строки, типа eval ('ЗаписатьВЛог(пСтрока)').

Причины минимизации:

1. Не наглядный.
2. Трудно отлаживать.
3. Скорее всего можно написать без eval.

В мире JavaScript разработчиков фраза: "Eval is evil" (eval — это зло)

Документирование

Пишите к коду комментарии в стиле [jsdoc](#).

При создании новых операций, функций такие комментарии предлагаются автоматически.

Комментарии раскрывают смысл кода

Полезно при использовании математических, экономических формул, которые рядовому программисту, сопровождающему непонятны или неизвестны.

Отсутствие незавершенного кода

Отметить TODO или удалить.